



UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO

COLEGIADO DE ENGENHARIA DA COMPUTAÇÃO

João Vitor Lopes Miranda

RELATÓRIO FINAL - ROBÓTICA

JUAZEIRO - BA

2026

ÍNDICE

1. INTRODUÇÃO.....	2
2. ARQUITETURA GERAL DO SISTEMA.....	3
3. MAPEAMENTO DO AMBIENTE.....	4
4. PLANEJAMENTO GLOBAL - A*	5
4.1. Construção do Grid de Ocupação.....	6
4.2. Modelo de Movimento Expandido.....	6
4.3. Heurística Adaptativa por Densidade de Obstáculos.....	6
4.4 Busca por Fila de Prioridade.....	6
4.5 Simplificação da Rota.....	6
5. CONTROLE LOCAL - DWA.....	8
5.1 Prioridade 1 - Desvio Frontal de Emergência.....	8
5.2 Prioridade 1.5 - Centralização Suave entre Paredes.....	8
5.3 Prioridade 2 — Confiança no Mapa Global.....	8
5.4 Prioridade 3 — Alinhamento em Passagens Estreitas.....	8
5.5 Prioridade 4 — Busca DWA Tradicional.....	9
5.6 Salvaguarda Absoluta de Emergência.....	9
6. RESULTADOS EXPERIMENTAIS.....	10
6.1 Mapeamento do Ambiente.....	10
6.2 Rota Planejada pelo A*	10
6.3 Trajetória Real Executada.....	11
6.4 Análise Quantitativa do Desvio.....	12
7. DISCUSSÃO.....	13
8. CONCLUSÃO.....	14

1. INTRODUÇÃO

Este relatório documenta o projeto de navegação autônoma de um robô móvel diferencial simulado no CoppeliaSim, integrando um planejador de rota global baseado no algoritmo A* com um controlador reativo local baseado no Dynamic Window Approach (DWA). O sistema também incorpora mapeamento do ambiente por visão computacional, usando uma câmera ortográfica para gerar o grid de ocupação a partir do qual o A* planeja a rota.

O objetivo central é permitir que o robô percorra, de forma autônoma e seguindo restrições cinemáticas realistas, o caminho entre uma posição inicial e um alvo (Target), desviando de obstáculos estáticos mapeados e de obstáculos dinâmicos não previstos, enquanto mantém suavidade de movimento (evitando comandos bruscos de velocidade angular).

A arquitetura combina, portanto, duas escalas de decisão complementares: uma camada de planejamento de alto nível (global, baseada em mapa) e uma camada de controle reativo de baixo nível (local, baseada em sensores), abordagem clássica e amplamente utilizada em robótica móvel para conciliar otimalidade de caminho com segurança em tempo real.

2. ARQUITETURA GERAL DO SISTEMA

O código está organizado em três componentes principais, cada um encapsulado em uma classe:

- **GlobalPlanner** — constrói o grid de ocupação a partir da nuvem de pontos de obstáculos e executa a busca A^* para encontrar a rota de menor custo até o alvo;
- **VisionMapper** — captura a imagem do sensor de visão ortográfico do CoppeliaSim, binariza a imagem e converte os pixels de obstáculo em coordenadas do mundo real (nuvem de pontos ox, oy);
- **LocalNavigator** — a cada passo de simulação, decide as velocidades linear (v) e angular (w) do robô, arbitrando entre cinco camadas de comportamento em ordem de prioridade.

A função `main()` orquestra o ciclo completo: conecta-se ao CoppeliaSim via ZeroMQ Remote API, obtém as posições inicial e do alvo, aciona o mapeamento por visão, planeja a rota global, simplifica essa rota em submetas (waypoints), e então entra no laço de controle em tempo real que aciona o LocalNavigator a cada passo de simulação (`client.step()`) até que o robô atinja o destino final.

3. MAPEAMENTO DO AMBIENTE

O mapeamento do ambiente é feito pela classe VisionMapper, que utiliza um sensor de visão ortográfico posicionado acima da cena (vista de topo). O processo de mapeamento segue as seguintes etapas:

- Captura da imagem RGB através de `sim.getVisionSensorImg`;
- Conversão para escala de cinza (luminância média dos três canais RGB);
- Binarização por limiar (threshold), classificando como obstáculo todo pixel cuja luminância ultrapassa 245 — ou seja, os elementos claros/brancos da cena, como paredes e barreiras;
- Conversão da posição de cada pixel ocupado para coordenadas cartesianas do mundo real, usando a posição da câmera e o tamanho da área ortográfica capturada (`ortho_size`);
- Descarte de pontos muito próximos à posição inicial do robô (raio de 0,35 m), evitando que o próprio robô seja interpretado como obstáculo.

A saída desse processo é uma nuvem de pontos (`ox`, `oy`) representando todos os obstáculos estáticos visíveis do topo, além da resolução por pixel (usada posteriormente para dimensionar o grid do A*).

4. PLANEJAMENTO GLOBAL - A*

A classe `GlobalPlanner` implementa uma variante do algoritmo A* clássico, com três diferenciais em relação à formulação tradicional: um modelo de movimento estendido, uma heurística adaptativa sensível à densidade de obstáculos, e uma etapa de simplificação geométrica do caminho resultante.

4.1. Construção do Grid de Ocupação

A partir da nuvem de pontos de obstáculos (ox , oy) fornecida pelo `VisionMapper`, o método `_build_occupancy_grid` discretiza o espaço em uma grade regular cujo tamanho de célula é definido por `cell_size`. Cada célula é marcada como ocupada se estiver a uma distância menor ou igual ao raio de inflação (`inflate_radius`) de algum ponto de obstáculo — este raio corresponde, na prática, ao raio físico do robô, garantindo margem de segurança nas rotas planejadas.

4.2. Modelo de Movimento Expandido

Diferentemente do A* tradicional com 8 direções (ortogonais e diagonais), o `MOVE_SET` desta implementação inclui 16 direções: as 8 direções convencionais mais 8 movimentos em "salto de cavalo" (deslocamentos como (2,1), (1,2), etc., com custo $\sqrt{5}$). Esse modelo de movimento estendido permite trajetórias mais suaves e com menor quantidade de mudanças bruscas de direção, aproximando melhor a rota discreta do grid de uma trajetória contínua.

4.3. Heurística Adaptativa por Densidade de Obstáculos

A heurística padrão do A* (distância euclidiana até o objetivo) é ponderada por um fator que reflete a densidade local de obstáculos ao redor de cada nó candidato (`_local_obstacle_ratio`, calculada em uma janela 7×7 células). Esse fator varia entre 0,5 e 1,5: regiões mais congestionadas recebem peso heurístico menor (a busca se torna mais "cautelosa", equivalente a um comportamento mais próximo da busca de custo uniforme, explorando mais alternativas), enquanto regiões livres recebem peso maior (a busca se torna mais "gulosa", convergindo mais rápido ao objetivo). Esse mecanismo é inspirado em formulações de A* adaptativo utilizadas em ambientes com obstáculos heterogêneos.

4.4 Busca por Fila de Prioridade

A busca em si (`compute_path`) segue a estrutura padrão do A*, mas utiliza o módulo `heapq` do Python como fila de prioridade (min-heap), em vez de uma varredura linear sobre um dicionário — trazendo complexidade $O(\log n)$ por inserção/remoção em vez de $O(n)$. Cada nó expandido é avaliado por $f(n) = g(n) + h_{\text{adaptativo}}(n)$, e o algoritmo termina assim que o nó-objetivo é retirado da fila.

4.5 Simplificação da Rota

A rota bruta retornada pelo A* segue estritamente as células do grid, resultando em um caminho serrilhado com muitos pontos. O método `simplify_path` realiza duas etapas de poda:

- Filtragem por variação angular: pontos são mantidos apenas quando a direção do caminho muda mais que 15° em relação ao segmento anterior (`_filter_by_turn_angle`), eliminando pontos redundantes em trechos retos;
- Poda por linha de visada (Line-of-Sight): entre os pontos remanescentes, o algoritmo tenta conectar diretamente pontos não consecutivos sempre que o segmento reto entre eles não cruza nenhum obstáculo (`_line_of_sight_clear`, verificado por amostragem a cada 0,15 m), reduzindo ainda mais o número de submetas.

O resultado final (`key_x`, `key_y`) é uma sequência enxuta de waypoints que preserva a validade geométrica da rota original, mas que é muito mais eficiente de seguir pelo controlador local, pois reduz o número de transições entre submetas.

5. CONTROLE LOCAL - DWA

A classe LocalNavigator é responsável pela geração dos comandos de velocidade linear (v) e angular (w) do robô a cada passo de simulação. Ao invés de aplicar o DWA de forma isolada, a implementação organiza o controle em cinco camadas de prioridade, avaliadas em sequência a cada chamada do método `step()`. Isso reflete a ideia de que, em navegação real, certas situações (como um obstáculo bem à frente) exigem reação imediata e não devem depender da otimização por janela dinâmica, que é mais adequada para espaços abertos.

5.1 Prioridade 1 - Desvio Frontal de Emergência

Quando o sensor frontal (SENSOR_MEIO) detecta um obstáculo a menos de 0,25 m, o robô reduz drasticamente a velocidade linear (0,04–0,05 m/s) e gira em direção ao lado com maior espaço livre, com maior intensidade angular (até 0,5 rad/s) caso os sensores diagonais também indiquem perigo ($< 0,15$ m) em ambos os lados.

5.2 Prioridade 1.5 - Centralização Suave entre Paredes

Em corredores, quando a diferença entre as distâncias laterais esquerda/direita é significativa ($> 0,12$ m) mas não há obstáculo frontal, o robô entra em modo de centralização, aplicando uma correção angular proporcional ao erro lateral (controlador proporcional simples), suavizada por um filtro passa-baixa que limita a taxa de variação angular ($\max_dw$). Esse modo é desativado automaticamente quando o robô se recentraliza ($\text{gap} < 0,08$ m) ou após 30 ciclos de controle.

5.3 Prioridade 2 — Confiança no Mapa Global

Quando os obstáculos detectados pelos sensores de proximidade já constavam do grid de ocupação construído pelo A* (ou seja, não há surpresas — obstáculo "conhecido"), o robô segue a submeta atual com controle proporcional de orientação, acelerando quando o erro de heading é pequeno ($< 0,5$ rad) e reduzindo a velocidade para girar no lugar quando o erro é grande. Essa camada representa a "confiança" do controlador local na rota pré-calculada pelo A*.

5.4 Prioridade 3 — Alinhamento em Passagens Estreitas

Quando ambos os lados apresentam obstáculos próximos ($< 0,25$ m) mas a frente está livre — configuração típica de uma passagem estreita entre dois obstáculos — o robô aplica uma correção angular suave e conservadora (limitada a $\pm 0,15$ rad/s) para atravessar o vão alinhado com o objetivo local, evitando colisões laterais.

5.5 Prioridade 4 — Busca DWA Tradicional

Na ausência de qualquer uma das condições acima (ambiente aberto, sem paredes próximas ou obstáculos conhecidos), o controlador recorre ao DWA clássico: gera uma janela

de combinações admissíveis de (v, w) — respeitando os limites de aceleração máxima (max_accel_v , max_accel_w) — simula a posição futura do robô para cada combinação (horizonte de 0,8 s) e avalia cada trajetória candidata por uma função de custo ponderada:

Parâmetro	Valor	Descrição
score_heading	peso 4,0	Alinhamento entre a orientação futura prevista e a direção do objetivo local
score_clearance	peso 0,8	Distância livre até o obstáculo mais próximo, com penalidades adicionais se a trajetória aproxima o robô de um lado já crítico
score_velocity	peso 2,0	Incentivo à velocidade mais alta quando o caminho está livre e bem alinhado

A combinação (v, w) de maior pontuação total é escolhida como comando de saída, sempre suavizada pelo mesmo filtro passa-baixa angular usado nas demais camadas, para evitar mudanças bruscas de direção entre ciclos de controle consecutivos.

5.6 Salvaguarda Absoluta de Emergência

Independentemente da camada ativa, se a menor distância detectada por qualquer sensor cair abaixo de 0,10 m, o sistema força uma manobra de recuo ($v = -0,02$ m/s) associada a um giro acentuado ($\pm 0,6$ rad/s) em direção ao lado com mais espaço, como último recurso antes de uma colisão.

6. RESULTADOS EXPERIMENTAIS

Esta seção apresenta os resultados obtidos em uma execução completa da simulação, do mapeamento inicial até a chegada do robô ao destino.

6.1 Mapeamento do Ambiente

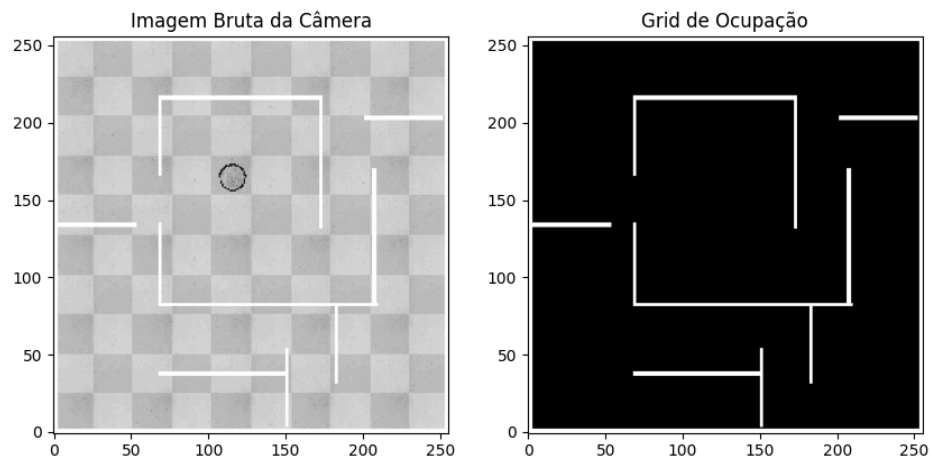


Figura 1 — Imagem bruta capturada pela câmera ortográfica (esquerda) e grid de ocupação binarizado (direita), utilizado como base para o planejamento A*.

A figura acima mostra o resultado do processo de mapeamento descrito na Seção 3. À esquerda, a imagem de luminância bruta capturada pela câmera; à direita, o resultado da binarização (threshold > 245), evidenciando claramente os contornos das paredes e obstáculos do ambiente, incluindo o pequeno círculo tracejado que representa o próprio robô (excluído do cálculo de obstáculos pelo raio de exclusão de 0,35 m).

6.2 Rota Planejada pelo A*

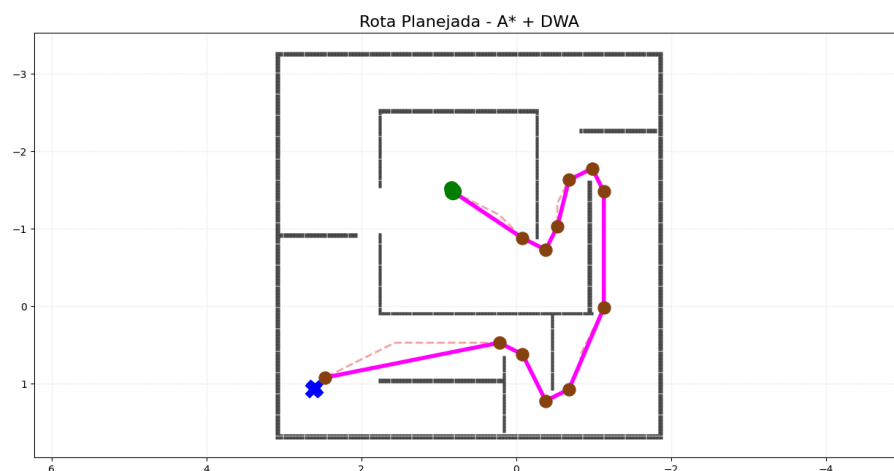


Figura 2 — Rota planejada pelo A*: caminho bruto do grid (linha tracejada rosa) e caminho simplificado por poda angular + linha de visada (linha magenta, com waypoints em marrom).

Nota-se que a rota simplificada (magenta) reduz significativamente o número de pontos em relação à rota bruta (tracejada), mantendo-se sempre dentro dos corredores livres do mapa, contornando as paredes internas do ambiente. O ponto verde indica a posição inicial do robô e o X azul indica a posição do alvo.

6.3 Trajetória Real Executada

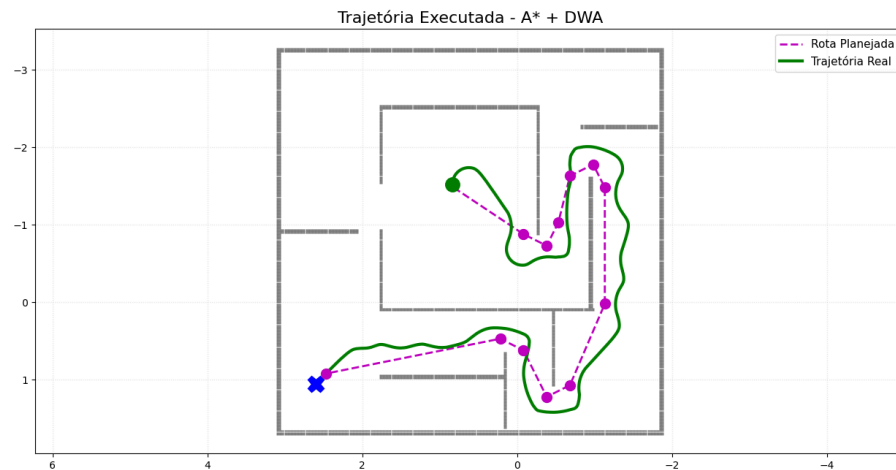


Figura 3 — Comparação entre a rota planejada (tracejado magenta) e a trajetória real executada pelo robô (linha verde contínua).

A trajetória real (verde) acompanha de perto a rota planejada, mas apresenta oscilações visíveis, especialmente nos trechos correspondentes às camadas reativas de centralização entre paredes e desvio de obstáculos — comportamento esperado do controlador local, que prioriza segurança e suavidade de movimento em detrimento da aderência estrita ao caminho ótimo do A*.

6.4 Análise Quantitativa do Desvio

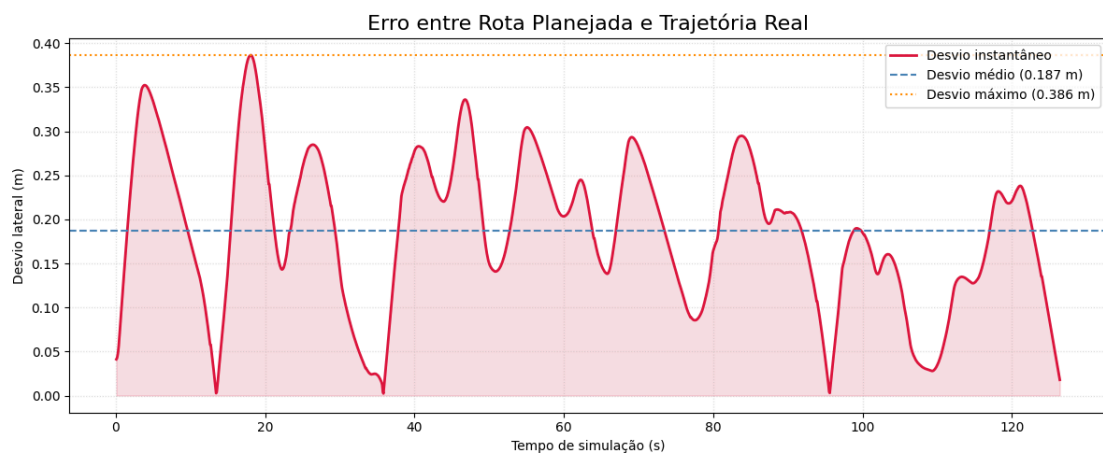


Figura 4 — Desvio lateral entre a trajetória real e a rota planejada ao longo do tempo de simulação.

O gráfico de desvio, calculado como a menor distância entre cada ponto da trajetória real e o segmento de reta mais próximo da rota planejada, evidencia um padrão oscilatório: picos de desvio (chegando a aproximadamente 0,386 m) coincidem com os trechos de curva mais acentuada da rota (onde o robô passa pelas camadas de centralização e desvio reativo), enquanto os vales de baixo desvio (próximos de 0,02–0,05 m) ocorrem nos trechos retos, onde a camada de "confiança no mapa global" (Prioridade 2) domina o controle. O desvio médio observado foi de aproximadamente 0,187 m.

7. DISCUSSÃO

Os resultados indicam que a arquitetura híbrida cumpriu o papel esperado: o A* forneceu uma rota globalmente coerente e livre de obstáculos conhecidos, enquanto o DWA e as camadas reativas absorveram as imprecisões locais — tanto por limitações do

sensoriamento de proximidade quanto pela necessidade de manter distância segura de paredes e obstáculos durante a execução.

O padrão de desvio observado na Figura 4 é consistente com o comportamento esperado de um controlador que arbitra entre múltiplas camadas de prioridade: desvios maiores concentram-se nas transições entre submetas e nos trechos onde curvas mais fechadas do A* exigem correções angulares mais agressivas, ao passo que a política de suavização (filtro passa-baixa sobre w) evita que o robô siga a rota ponto a ponto com precisão milimétrica — um trade-off deliberado entre fidelidade ao caminho planejado e conforto/segurança de movimento.

Como pontos de melhoria futura, podem-se destacar: (i) o ajuste dos pesos da função de custo do DWA (α , β , γ) para reduzir o desvio médio sem comprometer a segurança; (ii) a substituição da simplificação de rota baseada em ângulo fixo (15°) por um critério adaptativo à geometria local do ambiente; e (iii) a incorporação de replanejamento dinâmico do A* caso obstáculos não mapeados bloqueiem permanentemente a rota.

8. CONCLUSÃO

O projeto implementou com sucesso uma arquitetura de navegação híbrida para um robô diferencial simulado, combinando planejamento global via A* com heurística adaptativa e controle local reativo via DWA multi-camadas. Os resultados experimentais confirmam que o robô foi capaz de mapear o ambiente por visão computacional, planejar uma rota eficiente até o alvo e executá-la com desvio lateral médio inferior a 20 cm, validando a abordagem para o cenário testado.

